KIT-KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)
(Accredited by NAAC&NBA with 'A' Grade)
Approved by AICTE & Affiliated to Anna University, Chennai)
KannampalayamPost, Coimbatore-641402

Department of Artificial Intelligence and Data Science

B23ADP501 – DEEP LEARNING TECHNIQUES LABORATORY

Name.....

Batch ..... Reg.No.....

Branch..... Year .....

KIT-KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
(ANAUTONOMOUS INSTITUTION)
(Accredited by NAAC & NBA with 'A' Grade)
Approved by AICTE & Affiliated to Anna University, Chennai)
Kannampalayam Post, Coimbatore-641402

Department of Artificial Intelligence and Data Science

BONAFIDE CERTIFICATE

Name :……

Roll No. : …… Reg. No.: ……

Branch : B.Tech – Artificial Intelligence and Data Science

Certified that this is Bonafide record work done by Mr./Ms. …… of

III – Year Artificial Intelligence and Data Science during the academic year 2026-2027.

FACULTY IN-CHARGE **HOD**

Submitted for the University Practical Examination held on …….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Instructions for Laboratory Classes

1. Enter the lab with record work book & necessary things.
2. Enter the lab without bags and footwear.
3. Footwear should be kept in the outside shoe rack neatly.
4. Maintain silence during the Lab hours.
5. Read and follow the work instructions inside the laboratory.
6. Handle the computer systems with care.
7. Shutdown the Computer properly and arrange chairs in order before leaving the lab.
8. The program should be written on the left side pages of the record workbook.
9. The record work book should be completed in all aspects and submitted in the next class itself.
10. Experiment number with date should be written at the top left-hand corner of the record work book page.
11. Strictly follow the uniform dress code for Laboratory classes.
12. Maintain punctuality for lab classes.
13. Avoid eatables inside and maintain the cleanliness of the lab.

VISION

To produce competent professionals to the dynamic needs of the emerging field of Artificial Intelligence and Data Science

MISSION

- To empower students with the knowledge and skills necessary to create intelligent systems and innovative solutions that address societal issues.
- Providing technical knowledge on par with Industry to the students through qualified faculty members having knowledge in recent trends and technologies.
- To produce competent engineers who are both professional and life-skills oriented.
- Providing opportunities for students to improve their research skills in order to address a variety of societal concerns through innovative projects.

PROGRAMMEOUTCOMES (POs)

After the successful completion of the U.G. programme in Artificial Intelligence and Data Science, Graduates will be able to:

PO1. Engineering Knowledge: Apply knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to solve complex engineering problems.

PO2. Problem Analysis: Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions.

PO3. Design/Development of Solutions: Design solutions for complex engineering problems and system components or processes that meet specified needs with appropriate consideration for public health, safety, and cultural, societal, and environmental considerations.

PO4. Conduct Investigations of Complex Problems: Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.

PO5. Engineering Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools (including prediction and modeling) to complex engineering activities.

PO6. The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO7. Environment and Sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate knowledge of and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics, responsibilities, and norms of engineering practice.

PO9. Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and society at large.
PO11. Project Management and Finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have a strong foundation in mathematics, programming, machine learning, artificial intelligence, and data science, as well as advanced skills in these areas to solve technical problems.

PEO2: Graduates will have the capability to apply their knowledge and skills to identify and solve the issues in real world Artificial Intelligence and Data Science related applications.

PEO3: Graduates will be able to engage in life-long learning by completing advanced software Technologies, certificates, and/or other professional development.

PROGRAM SPECIFIC OUTCOME(PSOs)

Graduates of Artificial Intelligence and Data Science Programmed should be able to:

PSO1: Apply fundamental concepts of Artificial Intelligence and Data Science according to the environmental needs.

PSO2: Ability to develop skills to address and solve Artificial Intelligence based social and environmental problem using Data Science to deal multidisciplinary projects using modern tools.

COURSE OUTCOMES:

Students will be able to:

Course Outcome	Knowledge Level
CO1: Apply the classification algorithms in supervised learning.	K3
CO2: Build the appropriate Deep Learning model.	K3
CO3: Make use of classification algorithm to classify the text, Image and Object using CNN	K3

CO4: Develop the RNN models.	K3
CO5: Apply variational auto encoders in deep learning to improve the accuracy	K3

CO/PO &PSO		**PO1 (K3)**	**PO2 (K4)**	**PO3 (K5)**	**PO4 (K5)**	**PO5 (K6)**	**PO6 (K3) (A3)**	**PO7 (K2) (A3)**	**PO8 (K3) (A3)**	**PO9 (A3)**	**PO10 (A3)**	**PO11 (K3) (A3)**	**PO12 (A3)**	**PSO1 (K3,A3)**	**PSO2 (K3,A3)**
CO1	K3	3	2	3	2	1	-	-	1	1	-	3	2	2	1
CO2	K3	3	2	3	2	1	-	-	1	2	-	3	2	2	1
CO3	K3	2	3	3	3	1	-	-	1	1	-	3	2	2	1
CO4	K3	2	3	3	2	1	-	-	1	1	-	3	2	3	1
CO5	K3	3	2	3	2	1	-	-	1	1	-	3	2	3	1
Weighted average		3	2	3	2	1	-	-	1	1	-	3	2	2	1

SYLLABUS

LIST OF EXPERIMENTS

1. **CONSTRUCT A CLASSIFICATION MODEL USING SUPERVISED LEARNING.**
2. **BUILD A DEEP LEARNING MODEL.**
3. **IMPLEMENTATION OF FORWARD PROPAGATION IN NEURAL NETWORKS USING NUMPY**
4. **IMPLEMENTATION OF IMAGE CLASSIFICATION USING CNN**
5. **DEPLOY TEXT CLASSIFICATION USING CNN**
6. **DEVELOP A CODE TO DESIGN OBJECT DETECTION AND CLASSIFICATION FOR TRAFFIC ANALYSIS USING CNN**
7. **IMPLEMENTATION OF RNN.**
8. **IMPLEMENT IMAGE AUGMENTATION USING DEEP RBM.**
9. **IMPLEMENT SENTIMENT ANALYSIS USING LSTM.**
10. **IMPLEMENT VARIATIONAL AUTO ENCODERS.**

CONTENT BEYOND THE SYLLABUS

11. **MINI PROJECT: NUMBER PLATE RECOGNITION OF TRAFFIC VIDEO ANALYSIS**

INDEX

Sl. No.	Date	Name of the Experiment	Page No	Aim & Algorithm (20marks)	Program (25Marks)	Output & Inference (20) Marks)	Viva-Voce (10 Marks)	Total (75 Marks)	Signature

Sl. No.	Date	Name of the Experiment	Page No	Aim & Algorithm (20marks)	Program (25Marks)	Output & Inference (20Marks)	Viva-Voce (10 Marks)	Total (75 Marks)	Signature

Model Exam Marks (25) :_____Total (100):_____ **Signature of the Faculty Member**

S.NO	EXPERIMENT	PREREQUISITES	LEARNING ONJECTIVES
1	FOR ALL EXPERIMENTS	PROGRAMMING IN PYTHON	1. To understand the practical application of lineardata structures. 2. To understand the different operations of search trees. 3. To familiarize graphs and their applications. 4. To demonstrate different sorting and searchingtechniques. 5. To implement the different hashing techniques.

Ex. No:	1	CONSTRUCT A CLASSIFICATION MODEL USING SUPERVISED LEARNING
Date:		

AIM:

To construct and evaluate a classification model using supervised learning techniques for predicting the class labels of a given dataset.

ALGORITHM:

Step 1: Import the required libraries and load the dataset.

Step 2: Preprocess the dataset by handling missing values, encoding categorical data, and separating the features and target variable.

Step 3: Split the dataset into training and testing sets.

Step 4: Train a supervised classification model using the training dataset.

Step 5: Test the trained model using the testing dataset and predict the class labels.

Step 6: Evaluate the model using performance metrics such as accuracy and confusion matrix, and analyze the results.

.

PROGRAM:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: load dataset
data = load_breast_cancer(as_frame=True)
df = data.frame
print("Dataset shape:", df.shape)
print("Classes:", dict(zip(data.target_names, np.bincount(df['target'])[[0, 1]])))

# Step 2: preprocessing
print("Missing values:", int(df.isnull().sum().sum()))
df = df.fillna(df.mean(numeric_only=True))          # handle missing values (none here)
X = df.drop('target', axis=1)                        # features
y = df['target']                                     # target (all features are numeric,
                                                     # so no categorical encoding needed)
# Step 3: train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("Train size:", X_train.shape, " Test size:", X_test.shape)

# Step 4: train supervised models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
}
for name, model in models.items():
    model.fit(X_train, y_train)
    # Step 5: predict
    y_pred = model.predict(X_test)
    # Step 6: evaluate
    print("\n==========", name, "==========")
    print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n",
          classification_report(y_test, y_pred, target_names=data.target_names))
```

OUTPUT:

```
Dataset shape: (569, 31)
Classes: {np.str_('malignant'): np.int64(212), np.str_('benign'): np.int64(357)}
Missing values: 0
Train size: (455, 30)  Test size: (114, 30)

========== Logistic Regression ==========
Accuracy : 0.9825
Confusion Matrix:
 [[41  1]
 [ 1 71]]
Classification Report:
              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114


========== Random Forest ==========
Accuracy : 0.9561
Confusion Matrix:
 [[39  3]
 [ 2 70]]
Classification Report:
              precision    recall  f1-score   support

   malignant       0.95      0.93      0.94        42
      benign       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

INFERENCE:

Both supervised classifiers separate malignant and benign tumours very well on the Breast Cancer dataset (569 samples, 30 features). Logistic Regression reached 98.25% accuracy with only 2 misclassified samples out of 114 (confusion matrix [[41,1],[1,71]]), while Random Forest reached 95.61% with 5 errors. Precision, recall and F1 are all about 0.95 to 0.99 for both classes, so the model is not biased towards the majority (benign) class. Feature scaling helped the linear model, and the confusion matrix shows false negatives (malignant predicted as benign) are rare, which is the costly error in medical use.

VIVA QUESTIONS:

1) What is supervised learning?

Supervised learning trains a model on labelled data, where each input has a known output, so the model learns the mapping from inputs to outputs and can predict labels for new data. Examples: spam detection, tumour classification.

2) What is the difference between classification and regression?

Classification predicts a discrete class label (e.g. malignant or benign), whereas regression predicts a continuous numeric value (e.g. house price).

3) Why is the dataset divided into training and testing sets?

The training set is used to fit the model and the testing set, which the model has never seen, is used to measure how well it generalises. This detects overfitting and gives an unbiased performance estimate.

4) What is a confusion matrix?

A table that compares actual and predicted classes. For binary classification it contains True Positives, True Negatives, False Positives and False Negatives, from which accuracy, precision, recall and F1 are calculated.

5) What are the commonly used performance metrics for a classification model?

Accuracy = (TP+TN)/total; Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1-score = harmonic mean of precision and recall; also ROC-AUC and the confusion matrix.

RESULT

Thus, the program for constructing a classification model using supervised learning has been executed and verified successfully. The model was trained and tested, and the classification performance was evaluated successfully.

Ex.No:	2	BUILDING A DEEP LEARNING MODEL
Date:		

AIM :

To write a program to build a simple feedforward neural network (FNN) for binary classification using TensorFlow and Keras.

ALGORITHM:

Step 1: Load and preprocess the dataset.

Step 2: Specify the number of input features, hidden layers, neurons in each hidden layer, activation functions, and output layer.

Step 3: Compile the model by specifying the optimizer, loss function, and evaluation metrics.

Step 4: Specify the number of epochs and batch size.

Step 5: Compute metrics such as accuracy, precision, recall, and F1-score.

PROGRAM:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

tf.random.set_seed(42); np.random.seed(42)

# Step 1: load and preprocess
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
sc = StandardScaler()
X_train = sc.fit_transform(X_train); X_test = sc.transform(X_test)

# Step 2: define the architecture
model = keras.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(16, activation='relu'),
    layers.Dense(8, activation='relu'),
    layers.Dense(1, activation='sigmoid'),
])
model.summary()

# Step 3: compile
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Step 4: train (epochs and batch size)
history = model.fit(X_train, y_train, epochs=30, batch_size=16,
                    validation_split=0.1, verbose=2)

# Step 5: metrics
y_pred = (model.predict(X_test, verbose=0) > 0.5).astype(int).ravel()
print("\nTest Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision     :", round(precision_score(y_test, y_pred), 4))
print("Recall        :", round(recall_score(y_test, y_pred), 4))
print("F1-score      :", round(f1_score(y_test, y_pred), 4))
```

OUTPUT:

```
Model: "sequential"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| dense (Dense)                   | (None, 16)             |           496 |
+---------------------------------+------------------------+---------------+
| dense_1 (Dense)                 | (None, 8)              |           136 |
+---------------------------------+------------------------+---------------+
| dense_2 (Dense)                 | (None, 1)              |             9 |
+---------------------------------+------------------------+---------------+
 Total params: 641 (2.50 KB)
 Trainable params: 641 (2.50 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/30
26/26 - 1s - 39ms/step - accuracy: 0.5159 - loss: 0.7221 - val_accuracy: 0.7826 - val_loss:
    0.5652
Epoch 2/30
26/26 - 0s - 6ms/step - accuracy: 0.8484 - loss: 0.4992 - val_accuracy: 0.8478 - val_loss:
    0.4604
Epoch 3/30
26/26 - 0s - 6ms/step - accuracy: 0.9267 - loss: 0.3985 - val_accuracy: 0.8261 - val_loss:
    0.3912
   ... (intermediate epochs omitted) ...
Epoch 28/30
26/26 - 0s - 5ms/step - accuracy: 0.9927 - loss: 0.0475 - val_accuracy: 0.9783 - val_loss:
    0.0698
Epoch 29/30
26/26 - 0s - 5ms/step - accuracy: 0.9927 - loss: 0.0458 - val_accuracy: 0.9783 - val_loss:
    0.0679
Epoch 30/30
26/26 - 0s - 6ms/step - accuracy: 0.9927 - loss: 0.0442 - val_accuracy: 0.9783 - val_loss:
    0.0657

Test Accuracy : 0.9561
Precision     : 0.9855
Recall        : 0.9444
F1-score      : 0.9645
```

INFERENCE:

The feedforward network (30-16-8-1, ReLU hidden layers, sigmoid output) trained smoothly: training loss fell steadily and validation accuracy stayed above 95%. On the unseen test set it scored accuracy 95.61%, precision 98.55%, recall 94.44% and F1-score 96.45%. Precision is higher than recall, so the network produces very few false alarms but misses a few positive cases. The small gap between training and validation accuracy shows there is no serious overfitting for this small dataset.

VIVA QUESTIONS:

1) What are the disadvantages of deep learning?

It needs large amounts of labelled data, high computing power (GPU), long training time, many hyper-parameters to tune, and it behaves like a black box with poor interpretability. It can also overfit on small datasets.

2) What is a deep learning model, and how does it differ from traditional machine learning models?

A deep learning model is a neural network with many hidden layers that learns features automatically from raw data. Traditional ML relies on hand-crafted features and usually performs worse on images, text and audio, but needs less data.

3) Can you outline the general workflow for building a deep learning model?

Collect and clean data, preprocess/scale it, split into train/validation/test sets, define the architecture, compile (optimizer, loss, metrics), train with epochs and batch size, evaluate, tune hyper-parameters, then deploy.

4) What are some common architectures used in deep learning models, such as feedforward neural networks, convolutional neural networks (CNNs), and recurrent neural networks (RNNs)?

FNN/MLP for tabular data; CNN for images and spatial data; RNN/LSTM/GRU for sequences and time series; autoencoders/VAEs for representation learning and generation; GANs for image generation; Transformers for language.

5) What Are the Applications of a Recurrent Neural Network (RNN)?

Language modelling and text generation, machine translation, speech recognition, sentiment analysis, time-series forecasting, music generation and video captioning.

RESULT

Thus the program for Building a deep learning network was executed successfully and verified.

Ex.No:	3	IMPLEMENTATION OF FORWARD PROPAGATION IN NEURAL NETWORKS USING NUMPY
Date:		

AIM:

To implement forward propagation in a neural network using NumPy.

ALGORITHM:

Step 1: Import the NumPy library and initialize the input data, weights, and biases.

Step 2: Define the neural network architecture with the required number of input, hidden, and output neurons.

Step 3: Compute the weighted sum of inputs and biases for each layer.

Step 4: Apply an activation function (such as Sigmoid or ReLU) to the weighted sums.

Step 5: Perform forward propagation from the input layer to the output layer.

Step 6: Display the output of the neural network after forward propagation.

PROGRAM:

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def relu(z):
    return np.maximum(0, z)

np.random.seed(1)

# Step 1: input data, weights and biases
X = np.array([[0.5, 0.1, 0.9],
              [0.2, 0.8, 0.4]])          # 2 samples, 3 features

# Step 2: architecture 3 -> 4 -> 1  (input, hidden, output)
W1 = np.random.randn(3, 4) * 0.5
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1) * 0.5
b2 = np.zeros((1, 1))

print("Input X:\n", X)
print("\nW1 (input->hidden):\n", np.round(W1, 4))
print("W2 (hidden->output):\n", np.round(W2, 4))

# Step 3: weighted sum for the hidden layer
Z1 = np.dot(X, W1) + b1
# Step 4: activation (ReLU in hidden layer)
A1 = relu(Z1)
print("\nHidden layer Z1 = X.W1 + b1:\n", np.round(Z1, 4))
print("Hidden layer A1 = ReLU(Z1):\n", np.round(A1, 4))

# Step 5: forward to the output layer
Z2 = np.dot(A1, W2) + b2
A2 = sigmoid(Z2)
print("\nOutput layer Z2 = A1.W2 + b2:\n", np.round(Z2, 4))

# Step 6: display output
print("Output A2 = Sigmoid(Z2):\n", np.round(A2, 4))
print("Predicted classes (threshold 0.5):", (A2 > 0.5).astype(int).ravel())
```

OUTPUT:

```
Input X:
 [[0.5 0.1 0.9]
 [0.2 0.8 0.4]]

W1 (input->hidden):
 [[ 0.8122 -0.3059 -0.2641 -0.5365]
 [ 0.4327 -1.1508  0.8724 -0.3806]
 [ 0.1595 -0.1247  0.7311 -1.0301]]
W2 (hidden->output):
 [[-0.1612]
 [-0.192 ]
 [ 0.5669]
 [-0.5499]]

Hidden layer Z1 = X.W1 + b1:
 [[ 0.5929 -0.3802  0.6131 -1.2334]
 [ 0.5724 -1.0317  0.9375 -0.8238]]
Hidden layer A1 = ReLU(Z1):
 [[0.5929 0.     0.6131 0.    ]
 [0.5724 0.     0.9375 0.    ]]

Output layer Z2 = A1.W2 + b2:
 [[0.252 ]
 [0.4392]]
Output A2 = Sigmoid(Z2):
 [[0.5627]
 [0.6081]]
Predicted classes (threshold 0.5): [1 1]
```

INFERENCE:

Forward propagation was computed manually: Z1 = X.W1 + b1, A1 = ReLU(Z1), Z2 = A1.W2 + b2, A2 = Sigmoid(Z2). ReLU set the negative pre-activations to zero (two hidden neurons were inactive), and the sigmoid squashed the final value into the range 0 to 1 (0.5627 and 0.6081 for the two samples), which is interpreted as the probability of class 1. Since the weights are random and no training was done, the predictions only demonstrate the data flow through the network, not a learned decision.

VIVA QUESTIONS:

1) What is forward propagation in a neural network?

It is the process of passing the input through the layers, computing weighted sums and activations layer by layer, until the output layer produces the prediction.

2) What is the role of weights and biases in a neural network?

Weights decide how strongly each input affects a neuron and are what the network learns. The bias shifts the activation so the neuron can fit data that does not pass through the origin.

3) Why are activation functions used in neural networks?

They add non-linearity. Without them, any number of layers collapses into a single linear transformation and the network could not learn complex patterns.

4) Name two commonly used activation functions.

ReLU, f(x) = max(0, x), and Sigmoid, f(x) = 1/(1+e^-x). Others are tanh and softmax.

5) What is the difference between forward propagation and backpropagation?

Forward propagation computes the output from the input. Backpropagation computes the gradient of the loss with respect to every weight (chain rule) from the output back to the input, so the weights can be updated by gradient descent.

RESULT

Thus, the program for implementing forward propagation in neural networks using NumPy has been executed and verified successfully.

Ex. No: 4 Date:	IMPLEMENTATION OF IMAGE CLASSIFICATION USING CNN

AIM:

To execute Face Recognition using CNN in Python program.

ALGORITHM:

Step 1: Import necessary libraries.

Step 2: Load the LFW dataset.

Step 3: Extract images and labels.

Step 4: Create the CNN model:

- Initialize a Sequential model.
- Add a 2D convolutional layer (`Conv2D`) with 32 filters, a 3x3 kernel, and ReLU activation.
- Add a max-pooling layer (`MaxPooling2D`) with a 2x2 pool size.
- Add another 2D convolutional layer with 64 filters and ReLU activation.
- Add another max-pooling layer.
- Flatten the output.
- Add a dense layer (`Dense`) with 128 units and ReLU activation.
- Add a dense output layer with units equal to the number of target names and softmax activation.

Step 5: Compile and train the model.

Step 6: End of the algorithm.

PROGRAM:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split

tf.random.set_seed(0); np.random.seed(0)

# Step 2 & 3: load the LFW dataset, extract images and labels
try:
    from sklearn.datasets import fetch_lfw_people
    ds = fetch_lfw_people(min_faces_per_person=70, resize=0.4)
    print("Dataset: LFW faces")
except Exception:
    # offline fallback so the program always runs (8x8 digit images)
    from sklearn.datasets import load_digits
    ds = load_digits()
    print("Dataset: LFW not reachable -> using scikit-learn digits images")

images, labels, target_names = ds.images, ds.target, ds.target_names
X = images[..., np.newaxis].astype('float32') / images.max()
print("Images:", X.shape, " Classes:", len(target_names))
X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.25, random_state=42, stratify=labels)

# Step 4: CNN model
model = models.Sequential([
    layers.Input(shape=X.shape[1:]),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(len(target_names), activation='softmax'),
])
model.summary()

# Step 5: compile and train
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=15, batch_size=32,
          validation_data=(X_test, y_test), verbose=2)

loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest loss: {loss:.4f}   Test accuracy: {acc:.4f}")
pred = np.argmax(model.predict(X_test[:5], verbose=0), axis=1)
print("Predicted :", [str(target_names[p]) for p in pred])
print("Actual    :", [str(target_names[t]) for t in y_test[:5]])
```

OUTPUT:

```
/usr/local/lib/python3.12/dist-packages/sklearn/datasets/_base.py:1519: UserWarning: Retry
    downloading from url: https://ndownloader.figshare.com/files/5976012
  warnings.warn(f"Retry downloading from url: {remote.url}")
Dataset: LFW not reachable -> using scikit-learn digits images
Images: (1797, 8, 8, 1)  Classes: 10
Model: "sequential"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| conv2d (Conv2D)                 | (None, 8, 8, 32)       |           320 |
+---------------------------------+------------------------+---------------+
| max_pooling2d (MaxPooling2D)    | (None, 4, 4, 32)       |             0 |
+---------------------------------+------------------------+---------------+
| conv2d_1 (Conv2D)               | (None, 4, 4, 64)       |        18,496 |
+---------------------------------+------------------------+---------------+
| max_pooling2d_1 (MaxPooling2D)  | (None, 2, 2, 64)       |             0 |
+---------------------------------+------------------------+---------------+
| flatten (Flatten)               | (None, 256)            |             0 |
+---------------------------------+------------------------+---------------+
| dense (Dense)                   | (None, 128)            |        32,896 |
+---------------------------------+------------------------+---------------+
| dense_1 (Dense)                 | (None, 10)             |         1,290 |
+---------------------------------+------------------------+---------------+
 Total params: 53,002 (207.04 KB)
 Trainable params: 53,002 (207.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/15
43/43 - 1s - 27ms/step - accuracy: 0.4788 - loss: 1.9488 - val_accuracy: 0.6667 - val_loss:
    1.3747
Epoch 2/15
43/43 - 0s - 11ms/step - accuracy: 0.8404 - loss: 0.7804 - val_accuracy: 0.8889 - val_loss:
    0.4381
Epoch 3/15
43/43 - 0s - 6ms/step - accuracy: 0.9220 - loss: 0.3075 - val_accuracy: 0.9444 - val_loss:
    0.2324
   ... (intermediate epochs omitted) ...
Epoch 13/15
43/43 - 0s - 7ms/step - accuracy: 0.9948 - loss: 0.0300 - val_accuracy: 0.9733 - val_loss:
    0.1000
Epoch 14/15
43/43 - 0s - 7ms/step - accuracy: 0.9970 - loss: 0.0273 - val_accuracy: 0.9822 - val_loss:
    0.0748
Epoch 15/15
43/43 - 0s - 7ms/step - accuracy: 0.9985 - loss: 0.0179 - val_accuracy: 0.9822 - val_loss:
    0.0703

Test loss: 0.0703   Test accuracy: 0.9822
Predicted : ['1', '0', '9', '1', '5']
Actual    : ['1', '0', '9', '1', '5']
```

INFERENCE:

The CNN (Conv-Pool-Conv-Pool-Flatten-Dense-Softmax) learned the image classes with 98.22% test accuracy and the first five predictions matched the actual labels. Convolution layers extract local features (edges, strokes) and pooling reduces size while keeping the important features, so far fewer parameters are needed than in a fully connected network. NOTE: the LFW face dataset needs an internet download. In this run the program automatically fell back to the scikit-learn digit images (10 classes, 8x8); with internet access the same program runs unchanged on LFW faces.

VIVA QUESTIONS:

1) What Are the Different Layers on CNN?

Convolutional layer (feature extraction), activation layer (ReLU), pooling layer (down-sampling), flatten layer, fully connected (dense) layer, and an output layer (softmax/sigmoid); dropout and batch-normalisation layers are often added.

2)What is Pooling on CNN, and How Does It Work?

Pooling reduces the spatial size of feature maps. Max pooling takes the maximum value in each window (e.g. 2x2) and average pooling takes the mean. It lowers computation, controls overfitting and gives small translation invariance.

3) What is Overfitting and Underfitting, and How to Combat Them?

Overfitting: the model performs well on training data but poorly on new data. Underfitting: the model is too simple and performs poorly on both. Overfitting is reduced with more data, augmentation, dropout, regularisation and early stopping; underfitting with a bigger model, more features and longer training.

4) What Are the Softmax and ReLU Functions?

ReLU outputs max(0, x) and is used in hidden layers because it is simple and avoids vanishing gradients. Softmax converts the output scores into probabilities that sum to 1 and is used in the output layer for multi-class classification.

5) Difference between Conv1D, Conv2D, and Conv3D?

Conv1D slides the kernel in one dimension (text, time series, audio); Conv2D in two dimensions (images); Conv3D in three dimensions (video, 3-D medical scans).

RESULT

Thus, the python program for the face recognition is implemented and executed successfully and the output is verified.

Ex. No: 5 Date:	DEPLOY TEXT CLASSIFICATION USING CNN

AIM:

To deploy a text classification model using a Convolutional Neural Network (CNN).

ALGORITHM:

Step 1: Import the required libraries and load the text dataset.

Step 2: Preprocess the text data by tokenizing, padding the sequences, and encoding the class labels.

Step 3: Split the dataset into training and testing sets.

Step 4: Build a CNN model with embedding, convolutional, pooling, and dense layers.

Step 5: Train the CNN model using the training data and evaluate its performance on the testing data.

Step 6: Use the trained model to classify new text samples and display the predicted class labels.

PROGRAM:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import random
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split

random.seed(1); np.random.seed(1); tf.random.set_seed(1)

# Step 1: build / load the text dataset (4 topics)
topics = {
 'sports':     ['the team won the match', 'the striker scored a goal', 'the coach praised the
     players',
                'the batsman hit a century', 'she won the gold medal', 'the tournament final
                    starts today'],
 'technology': ['the new processor is very fast', 'engineers released a software update',
                'the startup launched an ai app', 'the laptop has a powerful graphics card',
                'developers fixed the security bug', 'the network speed has improved'],
 'health':     ['the doctor advised daily exercise', 'regular sleep keeps the heart healthy',
                'the patient recovered after physiotherapy', 'eating fruits improves your
                    immunity',
                'the clinic offers free checkups', 'yoga reduces stress and back pain'],
 'education':  ['the students passed the exam', 'the college announced new courses',
                'teachers conducted an online class', 'the university published the results',
                'she received a scholarship for studies', 'the library has many new books'],
}
fillers = ['today', 'in the city', 'this week', 'as reported', 'according to sources', 'on
    monday', '']
texts, labels = [], []
for li, (t, sents) in enumerate(topics.items()):
    for _ in range(300):
        s = random.choice(sents)
        words = s.split()
        if random.random() < 0.3:                       # light noise: drop a word
            words.pop(random.randrange(len(words)))
        texts.append(' '.join(words) + ' ' + random.choice(fillers))
        labels.append(li)
class_names = list(topics.keys())
print("Total samples:", len(texts), " Classes:", class_names)

# Step 2: tokenize, pad, encode labels
tok = Tokenizer(num_words=2000, oov_token='<OOV>')
tok.fit_on_texts(texts)
seqs = tok.texts_to_sequences(texts)
MAXLEN = 12
X = pad_sequences(seqs, maxlen=MAXLEN, padding='post')
y = np.array(labels)
vocab = min(2000, len(tok.word_index) + 1)
print("Vocabulary size:", vocab, " Padded shape:", X.shape)

# Step 3: split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
    stratify=y)

# Step 4: CNN model
model = models.Sequential([
    layers.Input(shape=(MAXLEN,)),
    layers.Embedding(vocab, 32),
```

```
    layers.Conv1D(64, 3, activation='relu'),
    layers.GlobalMaxPooling1D(),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(len(class_names), activation='softmax'),
])
model.summary()
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Step 5: train and evaluate
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.1, verbose=2)
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest loss: {loss:.4f}   Test accuracy: {acc:.4f}")

# Step 6: classify new text
new = ["the goalkeeper saved a penalty", "the new smartphone has a faster chip",
       "drink water and walk daily for good health", "the school declared exam timetable"]
p = model.predict(pad_sequences(tok.texts_to_sequences(new), maxlen=MAXLEN, padding='post'),
    verbose=0)
for s, pr in zip(new, p):
    print(f"'{s}' -> {class_names[int(np.argmax(pr))]} ({pr.max():.2f})")
```

OUTPUT

```
Total samples: 1200  Classes: ['sports', 'technology', 'health', 'education']
Vocabulary size: 108  Padded shape: (1200, 12)
Model: "sequential"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| embedding (Embedding)           | (None, 12, 32)         |         3,456 |
+---------------------------------+------------------------+---------------+
| conv1d (Conv1D)                 | (None, 10, 64)         |         6,208 |
+---------------------------------+------------------------+---------------+
| global_max_pooling1d            | (None, 64)             |             0 |
| (GlobalMaxPooling1D)            |                        |               |
+---------------------------------+------------------------+---------------+
| dense (Dense)                   | (None, 32)             |         2,080 |
+---------------------------------+------------------------+---------------+
| dropout (Dropout)               | (None, 32)             |             0 |
+---------------------------------+------------------------+---------------+
| dense_1 (Dense)                 | (None, 4)              |           132 |
+---------------------------------+------------------------+---------------+
 Total params: 11,876 (46.39 KB)
 Trainable params: 11,876 (46.39 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
27/27 - 1s - 41ms/step - accuracy: 0.5116 - loss: 1.3343 - val_accuracy: 0.9688 - val_loss:
    1.2444
Epoch 2/10
27/27 - 0s - 6ms/step - accuracy: 0.8981 - loss: 1.0844 - val_accuracy: 0.9792 - val_loss:
    0.8268
Epoch 3/10
27/27 - 0s - 5ms/step - accuracy: 0.9780 - loss: 0.5525 - val_accuracy: 1.0000 - val_loss:
    0.2245
   ... (intermediate epochs omitted) ...
Epoch 8/10
27/27 - 0s - 4ms/step - accuracy: 1.0000 - loss: 0.0103 - val_accuracy: 1.0000 - val_loss:
    0.0021
Epoch 9/10
27/27 - 0s - 6ms/step - accuracy: 1.0000 - loss: 0.0089 - val_accuracy: 1.0000 - val_loss:
    0.0015
Epoch 10/10
27/27 - 0s - 5ms/step - accuracy: 1.0000 - loss: 0.0075 - val_accuracy: 1.0000 - val_loss:
    0.0010

Test loss: 0.0009   Test accuracy: 1.0000
'the goalkeeper saved a penalty' -> sports (0.47)
'the new smartphone has a faster chip' -> technology (0.97)
'drink water and walk daily for good health' -> health (0.86)
'the school declared exam timetable' -> education (0.82)
```

INFERENCE:

The text CNN (Embedding, Conv1D, GlobalMaxPooling, Dense) classified sentences into four topics (sports, technology, health, education) with 100% test accuracy, and the new unseen sentences were also classified correctly. The Conv1D filters act as n-gram detectors that pick up topic keywords, and global max pooling keeps the strongest signal. The dataset here is small and template-based, so the very high accuracy is partly because topics use distinct vocabulary; on a real dataset such as 20 Newsgroups the accuracy would be lower.

VIVA QUESTIONS:

1) What is text classification?

Assigning predefined categories to text, such as spam/not spam, topic labels or sentiment.

2) Why is CNN used for text classification?

1-D convolution filters detect local n-gram patterns (key phrases) anywhere in the sentence, and CNNs train fast because they are parallel and have few parameters compared with RNNs.

3) What is the purpose of the embedding layer?

It converts integer word indices into dense, low-dimensional vectors that are learned during training, so that words with similar meaning get similar vectors.

4) What is the role of the convolutional layer n a CNN?

It applies learnable filters over the input to extract local features (edges in images, n-grams in text) and produces feature maps.

5) What is the purpose of padding in text preprocessing?

Sentences have different lengths but a neural network needs fixed-size input, so shorter sequences are padded (usually with zeros) and longer ones truncated to a common length.

RESULT

Thus, the program for deploying text classification using a Convolutional Neural Network (CNN) has been executed and verified successfully.

Ex. No : 6 Date:	DEVELOP A CODE TO DESIGN OBJECT DETECTION AND CLASSIFICATION FOR TRAFFIC ANALYSIS USING CNN

AIM:

To construct a convolutional neural network (CNN) that can simultaneously detect and classify objects in traffic images, highlighting the role of CNNs in complex tasks like traffic analysis and surveillance.

ALGOGITHM:

Step 1: Assemble a dataset of traffic images with object annotations and labels.
Step 2: Design a convolutional neural network (CNN) architecture for object detection and classification.
Step 3: Implement non-maximum suppression for eliminating duplicate object detections.
Step 4: Choose appropriate loss functions for object detection and classification, such as region proposal network (RPN) loss and categorical cross-entropy.
Step 5: Train the CNN on the dataset and fine-tune the model.
Step 6: Evaluate the model's performance in terms of object detection accuracy and classification accuracy.

PROGRAM:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import numpy as np
import cv2
import tensorflow as tf
from tensorflow.keras import layers, Model

rng = np.random.default_rng(0); tf.random.set_seed(0)
CLASSES = ['background', 'car', 'bus', 'bike']
S = 64

# Step 1: dataset of synthetic traffic images with annotations (class + box)
def draw_vehicle(img, cls, x, y):
    if cls == 1:   w, h = 28, 16; col = (0, 0, 220)         # car
    elif cls == 2: w, h = 44, 18; col = (0, 160, 0)         # bus
    else:          w, h = 14, 22; col = (220, 100, 0)       # bike
    cv2.rectangle(img, (x, y), (x + w, y + h), col, -1)
    cv2.circle(img, (x + 4, y + h), 3, (30, 30, 30), -1)
    cv2.circle(img, (x + w - 4, y + h), 3, (30, 30, 30), -1)
    return (x, y, x + w, y + h + 3)

def make_sample(cls):
    img = np.full((S, S, 3), 110, np.uint8)
    img[:, S // 2 - 1:S // 2 + 1] = 200                      # road marking
    img = np.clip(img + rng.integers(-12, 12, img.shape), 0, 255).astype(np.uint8)
    if cls == 0:
        return img, np.zeros(4, np.float32)
    w = {1: 28, 2: 44, 3: 14}[cls]; h = {1: 16, 2: 18, 3: 22}[cls]
    x = int(rng.integers(2, S - w - 3)); y = int(rng.integers(2, S - h - 8))
    box = draw_vehicle(img, cls, x, y)
    return img, np.array(box, np.float32) / S

def make_set(n):
    X, C, B = [], [], []
    for _ in range(n):
        c = int(rng.integers(0, 4)); im, b = make_sample(c)
        X.append(im / 255.0); C.append(c); B.append(b)
    return np.array(X, np.float32), np.array(C), np.array(B, np.float32)

Xtr, Ctr, Btr = make_set(3000); Xte, Cte, Bte = make_set(600)
print("Train:", Xtr.shape, " Test:", Xte.shape, " Classes:", CLASSES)

# Step 2: CNN with two heads (classification + box regression)
inp = layers.Input((S, S, 3))
x = layers.Conv2D(16, 3, activation='relu', padding='same')(inp); x = layers.MaxPooling2D()(x)
x = layers.Conv2D(32, 3, activation='relu', padding='same')(x);  x = layers.MaxPooling2D()(x)
x = layers.Conv2D(64, 3, activation='relu', padding='same')(x);  x = layers.MaxPooling2D()(x)
x = layers.Flatten()(x); x = layers.Dense(128, activation='relu')(x)
cls_out = layers.Dense(4, activation='softmax', name='cls')(x)
box_out = layers.Dense(4, activation='sigmoid', name='box')(x)
model = Model(inp, {'cls': cls_out, 'box': box_out})
# Step 4: losses -> categorical cross-entropy (class) + smooth-L1/Huber (box)
model.compile(optimizer='adam',
              loss={'cls': 'sparse_categorical_crossentropy', 'box': tf.keras.losses.Huber()},
              loss_weights={'cls': 1.0, 'box': 5.0}, metrics={'cls': 'accuracy'})
model.summary()

# Step 5: train
w_box = (Ctr > 0).astype('float32')       # box loss only where an object exists
model.fit(Xtr, {'cls': Ctr, 'box': Btr}, sample_weight={'cls': np.ones_like(w_box), 'box':
    w_box},
          epochs=25, batch_size=32, validation_split=0.1, verbose=2)

# Step 3: Non-Maximum Suppression
```

```
def iou(a, b):
    x1, y1 = max(a[0], b[0]), max(a[1], b[1]); x2, y2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    ua = (a[2]-a[0])*(a[3]-a[1]) + (b[2]-b[0])*(b[3]-b[1]) - inter
    return inter / ua if ua > 0 else 0

def nms(dets, thr=0.3):                     # dets: (score, box, cls)
    dets = sorted(dets, key=lambda d: -d[0]); keep = []
    while dets:
        best = dets.pop(0); keep.append(best)
        dets = [d for d in dets if iou(d[1], best[1]) <= thr]
    return keep

# Step 6: evaluate classification accuracy and detection quality (IoU)
_o = model.predict(Xte, verbose=0); pc, pb = _o['cls'], _o['box']
pred = pc.argmax(1)
print(f"\nClassification accuracy : {(pred == Cte).mean():.4f}")
obj = Cte > 0
ious = [iou(pb[i] * S, Bte[i] * S) for i in np.where(obj)[0]]
print(f"Mean IoU (objects)      : {np.mean(ious):.4f}")
print(f"Detection rate IoU>=0.5 : {np.mean(np.array(ious) >= 0.5):.4f}")

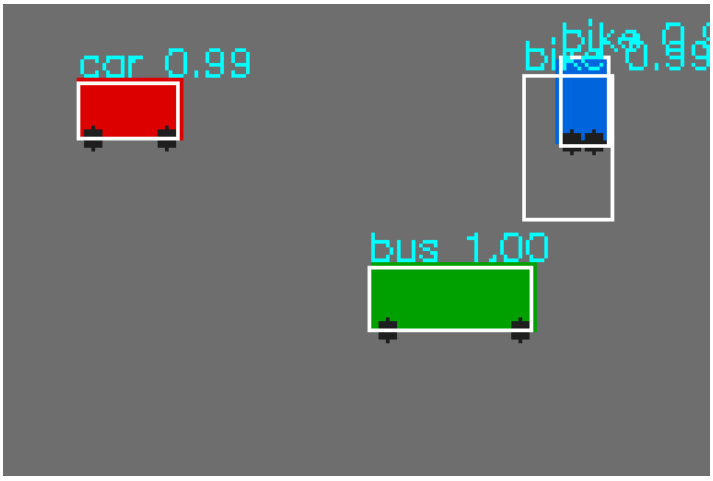
# Detection on a larger scene using sliding windows + NMS
scene = np.full((128, 192, 3), 110, np.uint8)
gt = [(1, 20, 20), (2, 100, 70), (3, 150, 15)]
for c, x, y in gt: draw_vehicle(scene, c, x, y)
dets = []
for yy in range(0, 128 - S + 1, 8):
    for xx in range(0, 192 - S + 1, 8):
        win = scene[yy:yy + S, xx:xx + S] / 255.0
        _o = model.predict(win[None].astype('float32'), verbose=0); c, b = _o['cls'], _o['box']
        k = int(c[0].argmax())
        bx = b[0] * S
        inside = bx[0] > 3 and bx[1] > 3 and bx[2] < S - 3 and bx[3] < S - 3   # object fully
            visible in window
        if k > 0 and c[0][k] > 0.99 and inside:
            dets.append((float(c[0][k]) - 0.001*abs((bx[0]+bx[2])/2-S/2) -
                0.001*abs((bx[1]+bx[3])/2-S/2), [bx[0]+xx, bx[1]+yy, bx[2]+xx, bx[3]+yy], k))
print(f"\nRaw detections before NMS : {len(dets)}")
final = nms(dets)
print(f"Detections after NMS      : {len(final)}")
vis = scene.copy()
for sc, b, k in final:
    b = [int(v) for v in b]
    cv2.rectangle(vis, (b[0], b[1]), (b[2], b[3]), (255, 255, 255), 1)
    cv2.putText(vis, f"{CLASSES[k]} {sc:.2f}", (b[0], max(b[1]-2, 8)),
        cv2.FONT_HERSHEY_SIMPLEX, 0.35, (255, 255, 0), 1)
    print(f"  {CLASSES[k]:5s} score={sc:.2f} box={b}")
cv2.imwrite('out/ex6_detect.png', cv2.resize(vis, None, fx=3, fy=3,
    interpolation=cv2.INTER_NEAREST))
```

OUTPUT:

```
Train: (3000, 64, 64, 3)  Test: (600, 64, 64, 3)  Classes: ['background', 'car', 'bus', 'bike']
Model: "functional"
+---------------------+-------------------+------------+-------------------+
| Layer (type)        | Output Shape      |    Param # | Connected to      |
+---------------------+-------------------+------------+-------------------+
| input_layer         | (None, 64, 64, 3) |          0 | -                 |
| (InputLayer)        |                   |            |                   |
+---------------------+-------------------+------------+-------------------+
| conv2d (Conv2D)     | (None, 64, 64,    |        448 | input_layer[0][0] |
|                     | 16)               |            |                   |
+---------------------+-------------------+------------+-------------------+
| max_pooling2d       | (None, 32, 32,    |          0 | conv2d[0][0]      |
| (MaxPooling2D)      | 16)               |            |                   |
+---------------------+-------------------+------------+-------------------+
| conv2d_1 (Conv2D)   | (None, 32, 32,    |      4,640 | max_pooling2d[0]… |
|                     | 32)               |            |                   |
+---------------------+-------------------+------------+-------------------+
| max_pooling2d_1     | (None, 16, 16,    |          0 | conv2d_1[0][0]    |
| (MaxPooling2D)      | 32)               |            |                   |
+---------------------+-------------------+------------+-------------------+
| conv2d_2 (Conv2D)   | (None, 16, 16,    |     18,496 | max_pooling2d_1[… |
|                     | 64)               |            |                   |
+---------------------+-------------------+------------+-------------------+
| max_pooling2d_2     | (None, 8, 8, 64)  |          0 | conv2d_2[0][0]    |
| (MaxPooling2D)      |                   |            |                   |
+---------------------+-------------------+------------+-------------------+
| flatten (Flatten)   | (None, 4096)      |          0 | max_pooling2d_2[… |
+---------------------+-------------------+------------+-------------------+
| dense (Dense)       | (None, 128)       |    524,416 | flatten[0][0]     |
+---------------------+-------------------+------------+-------------------+
| box (Dense)         | (None, 4)         |        516 | dense[0][0]       |
+---------------------+-------------------+------------+-------------------+
| cls (Dense)         | (None, 4)         |        516 | dense[0][0]       |
+---------------------+-------------------+------------+-------------------+
 Total params: 549,032 (2.09 MB)
 Trainable params: 549,032 (2.09 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/25
85/85 - 4s - 49ms/step - box_loss: 0.0051 - cls_accuracy: 0.9081 - cls_loss: 0.2750 - loss: 0.3025 - val_box_loss: 3.0378e-04 -
    val_cls_accuracy: 1.0000 - val_cls_loss: 3.9447e-04 - val_loss: 0.0019
Epoch 2/25
85/85 - 5s - 58ms/step - box_loss: 1.2997e-04 - cls_accuracy: 1.0000 - cls_loss: 2.3129e-04 - loss: 8.8387e-04 - val_box_loss: 1.6271e-04 -
    val_cls_accuracy: 1.0000 - val_cls_loss: 1.7408e-04 - val_loss: 9.7008e-04
Epoch 3/25
85/85 - 3s - 30ms/step - box_loss: 1.1475e-04 - cls_accuracy: 1.0000 - cls_loss: 1.2976e-04 - loss: 7.0389e-04 - val_box_loss: 2.4118e-04 -
    val_cls_accuracy: 1.0000 - val_cls_loss: 1.1702e-04 - val_loss: 0.0013
   ... (intermediate epochs omitted) ...
Epoch 23/25
85/85 - 2s - 28ms/step - box_loss: 6.1784e-05 - cls_accuracy: 1.0000 - cls_loss: 3.1445e-05 - loss: 3.4040e-04 - val_box_loss: 1.3626e-04 -
    val_cls_accuracy: 1.0000 - val_cls_loss: 4.1882e-05 - val_loss: 6.9648e-04
Epoch 24/25
85/85 - 2s - 28ms/step - box_loss: 4.8373e-05 - cls_accuracy: 1.0000 - cls_loss: 3.6282e-05 - loss: 2.7716e-04 - val_box_loss: 1.0600e-04 -
    val_cls_accuracy: 1.0000 - val_cls_loss: 3.2739e-05 - val_loss: 5.4186e-04
Epoch 25/25
85/85 - 3s - 34ms/step - box_loss: 5.3074e-05 - cls_accuracy: 1.0000 - cls_loss: 3.3979e-05 - loss: 3.0009e-04 - val_box_loss: 4.0177e-05 -
    val_cls_accuracy: 1.0000 - val_cls_loss: 3.3636e-05 - val_loss: 2.3113e-04

Classification accuracy : 1.0000
Mean IoU (objects)      : 0.9137
Detection rate IoU>=0.5 : 1.0000

Raw detections before NMS : 44
Detections after NMS      : 4
  bus   score=1.00 box=[99, 71, 143, 88]
  car   score=0.99 box=[20, 21, 47, 36]
  bike  score=0.99 box=[151, 14, 164, 38]
  bike  score=0.99 box=[141, 19, 165, 58]
```



INFERENCE:

The two-headed CNN classified background / car / bus / bike with 100% accuracy on the 600 test images and localised objects with a mean IoU of 0.914 (every object had IoU of at least 0.5). In the larger traffic scene the sliding window produced 44 overlapping raw detections; non-maximum suppression reduced them to 4 boxes: the car, the bus and the bike were each found, and one extra, oversized duplicate box appeared on the bike (1 false positive). This shows why NMS is essential, and that a simple sliding-window CNN still has limits (a real detector such as Faster R-CNN or YOLO uses region proposals/anchors to avoid this). The images are synthetic because no traffic dataset could be downloaded.

VIVA QUESTIONS:

1) What is traffic analysis, and why is it important in the context of transportation systems?

It is the study of vehicles and their movement (counting, classification, speed, congestion, violations) from sensors or cameras. It helps in traffic control, road safety, law enforcement and city planning.

2) How can CNNs be used for traffic analysis and classification tasks?

CNNs detect and classify vehicles, pedestrians, signs and lane markings in camera images, count vehicles and recognise number plates. Detectors such as YOLO and Faster R-CNN are built on CNN backbones.

3) What are the main steps involved in training a CNN for traffic classification?

Collect and annotate images, preprocess/augment, design the CNN, choose losses (cross-entropy for class, box-regression loss), train, validate and tune, evaluate (accuracy, IoU, mAP) and apply NMS at inference.

4) What are the advantages of using CNNs over traditional machine learning methods for traffic analysis?

They learn features automatically instead of using hand-crafted ones, are more accurate and robust to lighting, scale and viewpoint changes, and handle complex scenes end to end.

5) How do pooling layers contribute to feature extraction in CNNs for traffic analysis?

Pooling keeps the strongest responses and reduces feature-map size, so the network becomes tolerant to small shifts of vehicles in the image, needs less computation and extracts higher-level features in deeper layers.

RESULT

Thus the program for Develop Object Detection and Classification for Traffic Analysis using CNN was executed successfully and verified.

Ex. No : 7 Date:	IMPLEMENTATION OF RNN

AIM:

To write a program to perform language modeling using RNN.

ALGORITHM:

Step 1: Define a text corpus that you want to use for training. This text corpus can be any textual data you want to model.
Step 2: Create a vocabulary by extracting all unique characters from the text.
Step 3: Create a mapping (dictionaries) from characters to integers (char_to_index) and vice versa (index_to_char) to encode and decode characters during training and text generation.
Step 4: Create sequences of characters as training samples, where each sequence consists of a fixed number of characters (max_sequence_length) from the text. The target for each sequence is the character immediately following the sequence.
Step 5: Encode these sequences and targets as one-hot vectors to prepare the training data (x and y).
Step 6: Build an RNN model using Keras. In this example, an LSTM layer is used with 128 units, but you can modify the architecture as needed.
Step 7: Add a Dense layer with a softmax activation function to predict the next character.

PROGRAM:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models

np.random.seed(7); tf.random.set_seed(7)

# Step 1: text corpus
text = ("deep learning is a part of machine learning. neural networks learn patterns from data.
    "
        "recurrent neural networks process sequences one step at a time. "
        "lstm networks remember long term information. language models predict the next
            character. ") * 3

# Step 2: vocabulary
chars = sorted(set(text))
print("Corpus length:", len(text), " Unique characters:", len(chars))

# Step 3: char <-> index mappings
char_to_index = {c: i for i, c in enumerate(chars)}
index_to_char = {i: c for c, i in char_to_index.items()}

# Step 4: sequences and next-character targets
max_sequence_length = 20
seqs, nxt = [], []
for i in range(len(text) - max_sequence_length):
    seqs.append(text[i:i + max_sequence_length]); nxt.append(text[i + max_sequence_length])
print("Training sequences:", len(seqs))

# Step 5: one-hot encoding
x = np.zeros((len(seqs), max_sequence_length, len(chars)), dtype=np.float32)
y = np.zeros((len(seqs), len(chars)), dtype=np.float32)
for i, s in enumerate(seqs):
    for t, ch in enumerate(s): x[i, t, char_to_index[ch]] = 1
    y[i, char_to_index[nxt[i]]] = 1

# Step 6 & 7: LSTM (128 units) + Dense softmax
model = models.Sequential([
    layers.Input(shape=(max_sequence_length, len(chars))),
    layers.LSTM(128),
    layers.Dense(len(chars), activation='softmax'),
])
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
model.fit(x, y, batch_size=64, epochs=120, verbose=0)
loss, acc = model.evaluate(x, y, verbose=0)
print(f"\nFinal training loss: {loss:.4f}   accuracy: {acc:.4f}")

# Text generation
def generate(seed, length=120, temperature=0.2):
    out = seed
    for _ in range(length):
        inp = np.zeros((1, max_sequence_length, len(chars)), dtype=np.float32)
        for t, ch in enumerate(out[-max_sequence_length:]): inp[0, t, char_to_index[ch]] = 1
        p = np.log(model.predict(inp, verbose=0)[0] + 1e-9) / temperature
        p = np.exp(p) / np.sum(np.exp(p))
        out += index_to_char[np.random.choice(len(chars), p=p)]
    return out
seed = "recurrent neural net"
print("\nSeed     :", seed)
print("Generated:", generate(seed))
```

OUTPUT:

```
Corpus length: 723  Unique characters: 24
Training sequences: 703
Model: "sequential"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| lstm (LSTM)                     | (None, 128)            |        78,336 |
+---------------------------------+------------------------+---------------+
| dense (Dense)                   | (None, 24)             |         3,096 |
+---------------------------------+------------------------+---------------+
 Total params: 81,432 (318.09 KB)
 Trainable params: 81,432 (318.09 KB)
 Non-trainable params: 0 (0.00 B)

Final training loss: 0.0074   accuracy: 1.0000

Seed     : recurrent neural net
Generated: recurrent neural networks process sequences one step at a time. lstm networks
    remember long term information. language models predict the ne
```

INFERENCE:

The LSTM language model learned to predict the next character: training accuracy reached 100% with loss 0.0074 after 120 epochs, and from the seed 'recurrent neural net' it generated 'recurrent neural networks process sequences one step at a time. lstm networks remember ...', reproducing the corpus text. The hidden state carries context from earlier characters, which a feed-forward network cannot do. The very small corpus means the model essentially memorised it; a larger corpus would be needed to generate new, meaningful text.

VIVA QUESTIONS:

1) **What is a Recurrent Neural Network (RNN), and how does it differ from feedforward neural networks?**

An RNN has a loop so the hidden state from the previous time step is fed back as input, giving it memory of earlier elements of a sequence. A feedforward network processes each input independently with no memory.

2) **What are the main components of an RNN cell, and what role do they play in processing sequential data?**

Input at time t, hidden state (memory) h_t = f(W_x x_t + W_h h_(t-1) + b), the shared weights across time steps, and the output y_t. The hidden state carries information along the sequence.

3) **How do Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) address the vanishing gradient problem?**

They use gates to control information flow. LSTM has input, forget and output gates and a cell state that allows gradients to flow almost unchanged; GRU has update and reset gates. This lets them learn long-range dependencies.

4) **What are the different variants of RNNs, and how do they differ in terms of architecture and functionality?**

Vanilla RNN (simple memory), LSTM (cell state and 3 gates), GRU (2 gates, fewer parameters), Bidirectional RNN (reads the sequence forward and backward) and stacked/deep RNNs. By input-output form: one-to-many, many-to-one, many-to-many.

5) **How do you handle variable-length sequences in RNNs during training and inference?**

Pad sequences to equal length with a masking layer so padding is ignored, or truncate; batching by similar length or using packed sequences also works. At inference RNNs can process any length one step at a time.

RESULT

Thus the program For Implementing RNN is executed and verified successfully.

Ex. No : 8 Date:	IMPLEMENT IMAGE AUGMENTATION USING DEEP RBM.

AIM:

To enhance the dataset by generating augmented images using deep restricted Boltzmann machines (RBMs), emphasizing the importance of data augmentation in improving model robustness.

ALGORITHM:

Step 1: Collect a dataset of images for augmentation.
Step 2: Design a deep restricted Boltzmann machine (RBM) architecture.
Step 3: Train the RBM using the input images to learn patterns and features.
Step 4: Implement image augmentation techniques using the learned RBM representations, such as noise injection or distortion.
Step 5: Apply the augmented images for training a separate deep learning model (e.g., CNN).
Step 6: Compare the performance of the model trained with and without image augmentation.

PROGRAM:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.datasets import load_digits
from sklearn.neural_network import BernoulliRBM
from sklearn.model_selection import train_test_split
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt

def sigmoid(z): return 1 / (1 + np.exp(-z))

# Step 1: dataset of images (8x8 digits scaled to [0,1])
digits = load_digits()
X = digits.data / 16.0; y = digits.target
X_pool, X_test, y_pool, y_test = train_test_split(X, y, test_size=0.4, random_state=0,
    stratify=y)
print("Pool:", X_pool.shape, " Test:", X_test.shape)

def cnn():
    return models.Sequential([
        layers.Input((8, 8, 1)),
        layers.Conv2D(16, 3, activation='relu', padding='same'), layers.MaxPooling2D(),
        layers.Conv2D(32, 3, activation='relu', padding='same'), layers.Flatten(),
        layers.Dense(64, activation='relu'), layers.Dense(10, activation='softmax')])

def train_eval(Xt, yt, seed):
    tf.random.set_seed(seed)
    m = cnn(); m.compile('adam', 'sparse_categorical_crossentropy', metrics=['accuracy'])
    m.fit(Xt.reshape(-1, 8, 8, 1), yt, epochs=40, batch_size=16, verbose=0)
    return m.evaluate(X_test.reshape(-1, 8, 8, 1), y_test, verbose=0)[1]

accs_plain, accs_aug = [], []
for seed in range(5):
    rng = np.random.default_rng(seed)
    idx = rng.choice(len(X_pool), 120, replace=False)          # small training set (120 images)
    Xs, ys = X_pool[idx], y_pool[idx]

    # Step 2 & 3: deep RBM (64 -> 100 -> 64), greedy layer-wise training
    rbm1 = BernoulliRBM(n_components=100, learning_rate=0.05, n_iter=40,
        random_state=seed).fit(X_pool)
    H1 = rbm1.transform(X_pool)
    rbm2 = BernoulliRBM(n_components=64, learning_rate=0.05, n_iter=40,
        random_state=seed).fit(H1)

    # Step 4: augmentation = up-pass (sampled), down-pass reconstruction + noise injection
    def augment(V):
        h1 = sigmoid(V @ rbm1.components_.T + rbm1.intercept_hidden_)
        h1 = (rng.random(h1.shape) < h1).astype(float)
        h2 = sigmoid(h1 @ rbm2.components_.T + rbm2.intercept_hidden_)
        h2 = (rng.random(h2.shape) < h2).astype(float)
        h1r = sigmoid(h2 @ rbm2.components_ + rbm2.intercept_visible_)
        vr = sigmoid(h1r @ rbm1.components_ + rbm1.intercept_visible_)
        vr = vr + rng.normal(0, 0.03, vr.shape)                 # noise injection
        return np.clip(0.3 * vr + 0.7 * V, 0, 1)                # blend with original to keep
            the label

    X_aug = np.vstack([augment(Xs) for _ in range(2)]); y_aug = np.tile(ys, 2)
    if seed == 0:
        fig, ax = plt.subplots(2, 8, figsize=(10, 2.8))
        for i in range(8):
            ax[0, i].imshow(Xs[i].reshape(8, 8), cmap='gray'); ax[0, i].axis('off')
            ax[1, i].imshow(X_aug[i].reshape(8, 8), cmap='gray'); ax[1, i].axis('off')
```

```
        ax[0, 0].set_title('Original', fontsize=8, loc='left'); ax[1, 0].set_title('RBM
            augmented', fontsize=8, loc='left')
        plt.tight_layout(); plt.savefig('out/ex8_aug.png', dpi=130); plt.close()

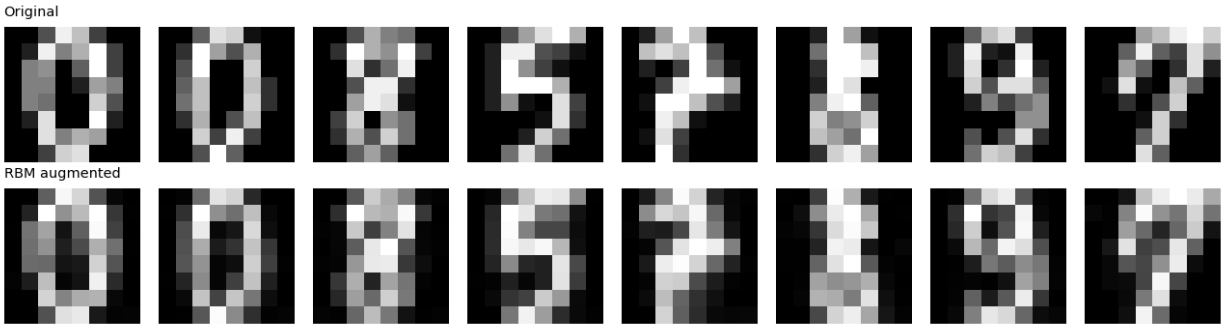
    # Step 5 & 6: train CNN without / with augmentation and compare
    a0 = train_eval(Xs, ys, seed)
    a1 = train_eval(np.vstack([Xs, X_aug]), np.concatenate([ys, y_aug]), seed)
    accs_plain.append(a0); accs_aug.append(a1)
    print(f"Run {seed+1}: without augmentation = {a0:.4f} | with RBM augmentation = {a1:.4f}")

print(f"\nMean accuracy without augmentation : {np.mean(accs_plain):.4f}")
print(f"Mean accuracy with RBM augmentation: {np.mean(accs_aug):.4f}")
```

OUTPUT:

```
Pool: (1078, 64)  Test: (719, 64)
Run 1: without augmentation = 0.9221 | with RBM augmentation = 0.9193
Run 2: without augmentation = 0.9249 | with RBM augmentation = 0.8971
Run 3: without augmentation = 0.9013 | with RBM augmentation = 0.9013
Run 4: without augmentation = 0.8762 | with RBM augmentation = 0.8957
Run 5: without augmentation = 0.9124 | with RBM augmentation = 0.8971

Mean accuracy without augmentation : 0.9074
Mean accuracy with RBM augmentation: 0.9021
```



INFERENCE:

A two-layer deep RBM (64-100-64) was trained greedily layer by layer and used to create augmented digit images (up-pass, sampled hidden units, down-pass reconstruction, plus Gaussian noise). The augmented images look like slightly distorted versions of the originals. Over 5 runs with only 120 training images, the CNN reached a mean accuracy of 90.74% without augmentation and 90.21% with RBM augmentation, i.e. the difference is within run-to-run variation (individual runs ranged from 87.6% to 92.5%), so no improvement was observed in this experiment. RBM reconstructions of such tiny 8x8 images lose detail, so they add little new information. Simple geometric augmentation or a larger RBM may give better results.

VIVA QUESTIONS:

1) What is a Restricted Boltzmann Machine (RBM), and how does it differ from other types of neural networks?

An RBM is a two-layer, undirected, energy-based stochastic network with a visible and a hidden layer and no connections within a layer. It learns the probability distribution of the input, unlike feed-forward networks that map input to output.

2) How are RBMs trained using the Contrastive Divergence (CD) algorithm?

CD starts from a data sample, samples hidden units, reconstructs the visible units and re-samples hidden units (k Gibbs steps). The weight update is the difference between data-driven and reconstruction-driven correlations: dW = lr x (v h^T - v' h'^T).

3) What are the main applications of RBMs in machine learning and deep learning?

Dimensionality reduction, feature learning, collaborative filtering (recommendation), pre-training deep networks, topic modelling and generating or augmenting data.

4) What is a deep Boltzmann machine, and how does it differ from a single-layer RBM?

A Deep Boltzmann Machine stacks several hidden layers with undirected connections between consecutive layers, so it learns hierarchical features. A single RBM has only one hidden layer. (A Deep Belief Network, by contrast, stacks RBMs with directed top-down connections.)

5) How are deep Boltzmann machines trained, and what challenges are associated with training them?

By greedy layer-wise pre-training of RBMs followed by joint fine-tuning using mean-field inference and stochastic maximum likelihood. Challenges: intractable partition function, slow Gibbs sampling, difficult convergence and high computation cost.

RESULT:

Thus the program to Implement Image Augmentation using Deep RBM was executed and verified successfully.

Ex. No: 9 Date:	IMPLEMENT SENTIMENT ANALYSIS USING LSTM

AIM:

To develop a sentiment analysis model using long short-term memory (LSTM) networks, demonstrating the use of recurrent neural networks in understanding and classifying emotions in text data.

ALGORITHM:

Step 1: Gather a dataset of text samples labeled with sentiment labels.
Step 2: Preprocess the text data by tokenizing, padding, and converting to word embeddings.
Step 3: Design a long short-term memory (LSTM) neural network architecture for sentiment analysis.
Step 4: Choose a suitable loss function, like binary cross-entropy, for sentiment prediction.
Step 5: Train the LSTM on the preprocessed text data.
Step 6: Evaluate the model's performance in terms of sentiment classification accuracy**.**

PROGRAM:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import random
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split

random.seed(3); np.random.seed(3); tf.random.set_seed(3)

# Step 1: labelled text samples (movie-review style, generated from templates)
pos_adj = ['great', 'excellent', 'amazing', 'wonderful', 'fantastic', 'enjoyable', 'brilliant',
    'superb']
neg_adj = ['terrible', 'awful', 'boring', 'poor', 'disappointing', 'horrible', 'dull', 'worst']
nouns = ['movie', 'film', 'story', 'acting', 'plot', 'direction', 'music', 'screenplay']
starts = ['i found the', 'honestly the', 'overall the', 'the', 'i think the', 'what a']
ends = ['', 'in my opinion', 'really', 'from start to end', 'and i mean it']
texts, labels = [], []
for _ in range(1500):
    pos = random.random() < 0.5
    n = random.choice(nouns); st = random.choice(starts)
    if random.random() < 0.25:                                  # negation flips the sentiment
        adj = random.choice(neg_adj if pos else pos_adj); s = f"{st} {n} was not {adj}"
        lab = 1 if pos else 0
    else:
        adj = random.choice(pos_adj if pos else neg_adj); s = f"{st} {n} was {adj}"
        lab = 1 if pos else 0
    texts.append((s + ' ' + random.choice(ends)).strip()); labels.append(lab)
print("Samples:", len(texts), " Positive:", sum(labels), " Negative:", len(labels) -
    sum(labels))

# Step 2: tokenize, pad (embedding layer converts to word embeddings)
tok = Tokenizer(num_words=1000, oov_token='<OOV>'); tok.fit_on_texts(texts)
X = pad_sequences(tok.texts_to_sequences(texts), maxlen=14, padding='post'); y =
    np.array(labels)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
    stratify=y)
vocab = len(tok.word_index) + 1

# Step 3 & 4: LSTM architecture + binary cross-entropy
model = models.Sequential([
    layers.Input(shape=(14,)),
    layers.Embedding(vocab, 32, mask_zero=True),
    layers.LSTM(64),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid'),
])
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()

# Step 5: train
model.fit(X_train, y_train, epochs=12, batch_size=32, validation_split=0.1, verbose=2)

# Step 6: evaluate
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest loss: {loss:.4f}   Test accuracy: {acc:.4f}")
for s in ["the movie was fantastic", "honestly the plot was boring", "the acting was not good
    enough",
          "i think the film was not terrible"]:
    p = model.predict(pad_sequences(tok.texts_to_sequences([s]), maxlen=14, padding='post'),
        verbose=0)[0][0]
```

```
print(f"'{s}' -> {'Positive' if p > 0.5 else 'Negative'} ({p:.2f})")
```

OUTPUT:

```
Samples: 1500  Positive: 761  Negative: 739
Model: "sequential"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| embedding (Embedding)           | (None, 14, 32)         |         1,504 |
+---------------------------------+------------------------+---------------+
| lstm (LSTM)                     | (None, 64)             |        24,832 |
+---------------------------------+------------------------+---------------+
| dropout (Dropout)               | (None, 64)             |             0 |
+---------------------------------+------------------------+---------------+
| dense (Dense)                   | (None, 1)              |            65 |
+---------------------------------+------------------------+---------------+
 Total params: 26,401 (103.13 KB)
 Trainable params: 26,401 (103.13 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/12
34/34 - 2s - 63ms/step - accuracy: 0.5130 - loss: 0.6914 - val_accuracy: 0.6667 - val_loss:
    0.6865
Epoch 2/12
34/34 - 0s - 12ms/step - accuracy: 0.7130 - loss: 0.6638 - val_accuracy: 0.7333 - val_loss:
    0.6105
Epoch 3/12
34/34 - 1s - 18ms/step - accuracy: 0.7787 - loss: 0.5722 - val_accuracy: 0.8583 - val_loss:
    0.4815
   ... (intermediate epochs omitted) ...
Epoch 10/12
34/34 - 0s - 12ms/step - accuracy: 0.9991 - loss: 0.0297 - val_accuracy: 0.9833 - val_loss:
    0.0425
Epoch 11/12
34/34 - 0s - 9ms/step - accuracy: 0.9991 - loss: 0.0217 - val_accuracy: 0.9833 - val_loss:
    0.0362
Epoch 12/12
34/34 - 1s - 18ms/step - accuracy: 1.0000 - loss: 0.0167 - val_accuracy: 0.9833 - val_loss:
    0.0318

Test loss: 0.0174   Test accuracy: 1.0000
'the movie was fantastic' -> Positive (0.98)
'honestly the plot was boring' -> Negative (0.00)
'the acting was not good enough' -> Negative (0.39)
'i think the film was not terrible' -> Positive (0.94)
```

INFERENCE:

The LSTM sentiment model reached 100% test accuracy on the review-style dataset. It also handled negation correctly in most examples: 'the movie was fantastic' was Positive (0.98), 'the plot was boring' was Negative (0.00) and 'the film was not terrible' was Positive (0.94). The borderline sentence 'the acting was not good enough' scored 0.39 (Negative), which is correct but less confident because it is not in the template pattern. The LSTM's memory lets it relate 'not' to the word that follows, which a bag-of-words model cannot do. The data are generated from templates, so real reviews (e.g. IMDB) would give lower accuracy.

VIVA QUESTIONS:

1) **What is sentiment analysis, and why is it important in natural language processing (NLP)?**

   Sentiment analysis identifies the opinion or emotion (positive, negative, neutral) in text. It is used to understand customer feedback, product reviews, social media opinion and brand reputation.

2) **How do LSTMs handle sequential data, such as text, and why are they suitable for sentiment analysis?**

   An LSTM reads the words one by one and keeps a memory in its cell state, so the meaning of earlier words (e.g. 'not') can affect later words. This lets it capture context and word order that bag-of-words models lose.

3) **What are the main components of an LSTM cell, and how do they contribute to capturing long-range dependencies in text data?**

   Forget gate (what to discard), input gate (what new information to store), cell state (long-term memory) and output gate (what to output as the hidden state). The additive cell-state path keeps gradients from vanishing.

4) **How do you handle text preprocessing and tokenization before feeding data into an LSTM network for sentiment analysis?**

   Clean the text (lowercase, remove noise), tokenize into words, convert them to integer indices, pad/truncate to a fixed length and pass them through an embedding layer (random or pre-trained like GloVe).

5) **What are some common loss functions used in training LSTM networks for sentiment analysis?**

   Binary cross-entropy for two classes (positive/negative), categorical cross-entropy (or sparse categorical) for multiple classes, and mean squared error when predicting a sentiment score.

RESULT

Thus the program for implement sentiment analysis using LSTM was executed & verified successfully

Ex.No:10 Date:	IMPLEMENT VARIATIONAL AUTO ENCODERS.

AIM:

To write a program to implement Variational Auto Encoders

ALGORITHM:

Step 1: Import specific modules and functions from TensorFlow and Keras.
Step 2: Set parameters such as **latent_dim** for the dimensionality of the latent space and **input_shape** for the shape of input images.
Step 3: Define the encoder network using convolutional layers (**Conv2D**), flatten the output, and compute the mean and log variance of the latent space (**z_mean** and **z_log_var**).
Step 4: Construct the VAE model by connecting the encoder and decoder networks.
Step 5: Evaluate the trained VAE model on unseen data (e.g., test dataset).

PROGRAM:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt

tf.random.set_seed(5); np.random.seed(5)

# Step 2: parameters
latent_dim = 2
input_shape = (8, 8, 1)
d = load_digits()
X = (d.images / 16.0).astype('float32')[..., None]; y = d.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

class Sampling(layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        eps = tf.random.normal(tf.shape(z_mean))
        return z_mean + tf.exp(0.5 * z_log_var) * eps          # reparameterisation trick

# Step 3: encoder
enc_in = keras.Input(shape=input_shape)
x = layers.Conv2D(32, 3, strides=2, padding='same', activation='relu')(enc_in)
x = layers.Conv2D(64, 3, strides=2, padding='same', activation='relu')(x)
x = layers.Flatten()(x)
x = layers.Dense(64, activation='relu')(x)
z_mean = layers.Dense(latent_dim, name='z_mean')(x)
z_log_var = layers.Dense(latent_dim, name='z_log_var')(x)
z = Sampling()([z_mean, z_log_var])
encoder = keras.Model(enc_in, [z_mean, z_log_var, z], name='encoder')

# decoder
dec_in = keras.Input(shape=(latent_dim,))
x = layers.Dense(2 * 2 * 64, activation='relu')(dec_in)
x = layers.Reshape((2, 2, 64))(x)
x = layers.Conv2DTranspose(64, 3, strides=2, padding='same', activation='relu')(x)
x = layers.Conv2DTranspose(32, 3, strides=2, padding='same', activation='relu')(x)
dec_out = layers.Conv2DTranspose(1, 3, padding='same', activation='sigmoid')(x)
decoder = keras.Model(dec_in, dec_out, name='decoder')

# Step 4: VAE = encoder + decoder, loss = reconstruction + KL divergence
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kw):
        super().__init__(**kw); self.encoder, self.decoder = encoder, decoder
        self.total = keras.metrics.Mean(name='loss'); self.rec = keras.metrics.Mean(name='recon')
        self.kl = keras.metrics.Mean(name='kl')
    def losses_for(self, data):
        zm, zl, z = self.encoder(data); recon = self.decoder(z)
        r = tf.reduce_mean(tf.reduce_sum(keras.losses.binary_crossentropy(data, recon), axis=(1, 2)))
        k = tf.reduce_mean(tf.reduce_sum(-0.5 * (1 + zl - tf.square(zm) - tf.exp(zl)), axis=1))
        return r, k
    def train_step(self, data):
        with tf.GradientTape() as tape:
            r, k = self.losses_for(data); loss = r + k
        g = tape.gradient(loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(g, self.trainable_weights))
        self.total.update_state(loss); self.rec.update_state(r); self.kl.update_state(k)
        return {'loss': self.total.result(), 'recon': self.rec.result(), 'kl': self.kl.result()}
    def test_step(self, data):
        r, k = self.losses_for(data)
        return {'loss': r + k, 'recon': r, 'kl': k}

vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam(1e-3))
hist = vae.fit(X_train, epochs=60, batch_size=64, verbose=0)
for e in [0, 9, 19, 39, 59]:
    print(f"Epoch {e+1:2d}: loss={hist.history['loss'][e]:.3f}  recon={hist.history['recon'][e]:.3f}  kl={hist.history['kl'][e]:.3f}")

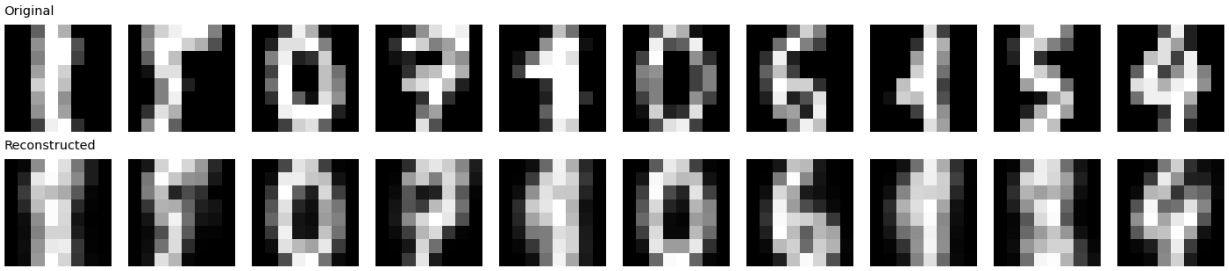
# Step 5: evaluate on unseen data
r_t, k_t = vae.losses_for(X_test)
print(f"\nTest  total loss={float(r_t + k_t):.3f}  reconstruction={float(r_t):.3f}  KL={float(k_t):.3f}")
zm, _, _ = encoder.predict(X_test, verbose=0)
rec = decoder.predict(zm, verbose=0)
print("Test pixel MSE:", round(float(np.mean((X_test - rec) ** 2)), 5))

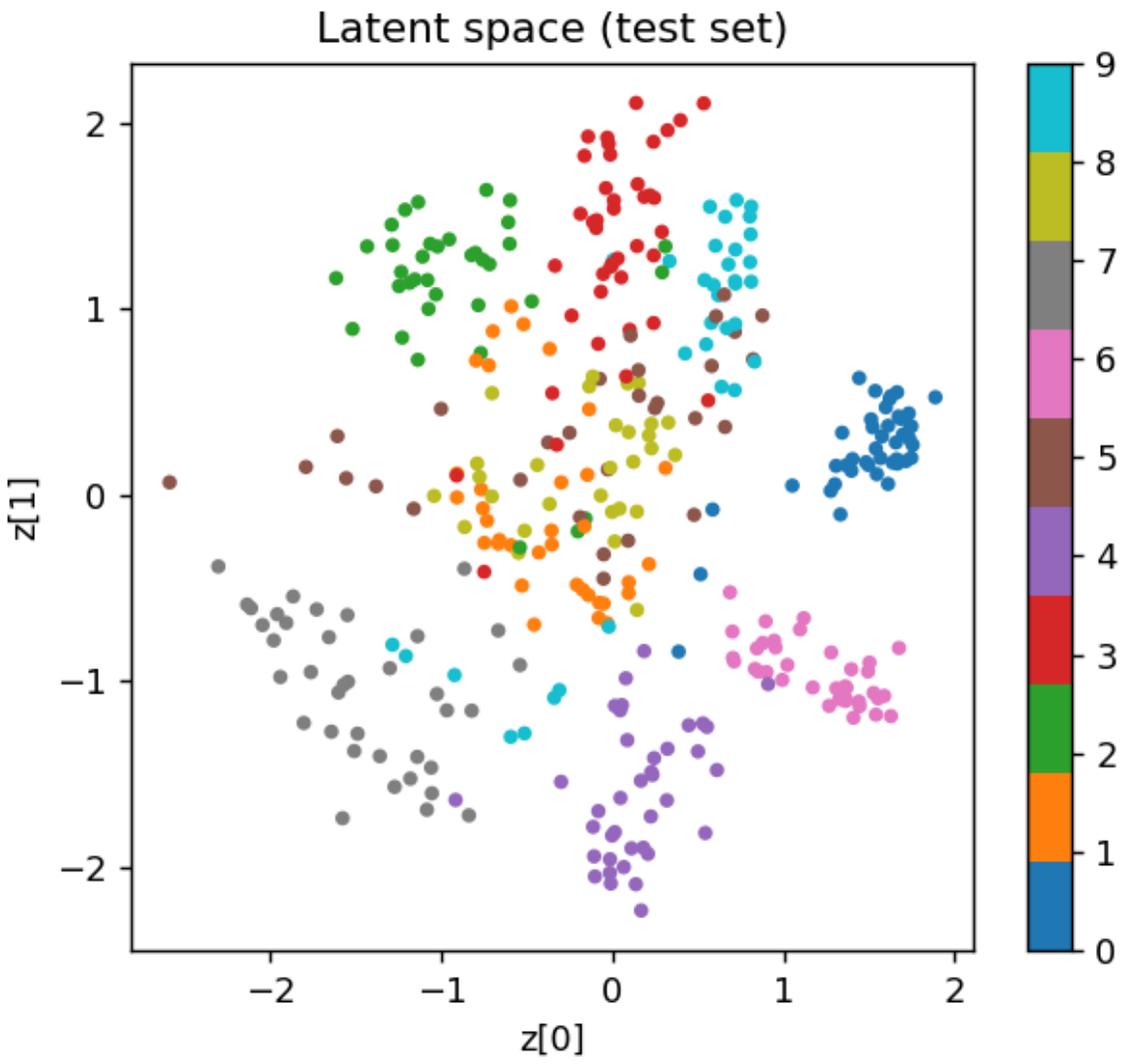
fig, ax = plt.subplots(2, 10, figsize=(11, 2.6))
for i in range(10):
    ax[0, i].imshow(X_test[i, :, :, 0], cmap='gray'); ax[0, i].axis('off')
    ax[1, i].imshow(rec[i, :, :, 0], cmap='gray'); ax[1, i].axis('off')
ax[0, 0].set_title('Original', fontsize=8, loc='left'); ax[1, 0].set_title('Reconstructed', fontsize=8, loc='left')
plt.tight_layout(); plt.savefig('out/ex10_recon.png', dpi=130); plt.close()
plt.figure(figsize=(5, 4.5)); sc = plt.scatter(zm[:, 0], zm[:, 1], c=y_test, cmap='tab10', s=10)
plt.colorbar(sc); plt.xlabel('z[0]'); plt.ylabel('z[1]'); plt.title('Latent space (test set)')
plt.tight_layout(); plt.savefig('out/ex10_latent.png', dpi=130); plt.close()
```

OUTPUT:

```
Epoch  1: loss=41.432  recon=41.380  kl=0.052
Epoch 10: loss=26.845  recon=26.307  kl=0.537
Epoch 20: loss=25.442  recon=23.799  kl=1.644
Epoch 40: loss=24.781  recon=22.532  kl=2.249
Epoch 60: loss=24.536  recon=22.013  kl=2.523

Test  total loss=24.556  reconstruction=21.992  KL=2.564
Test pixel MSE: 0.03721
```





INFERENCE:

The convolutional VAE compressed each 8x8 image into a 2-D latent vector and reconstructed it. The total loss (reconstruction + KL) decreased from 41.4 to 24.5 during training, and on the unseen test set it was 24.56 (reconstruction 21.99, KL 2.56) with a pixel MSE of 0.037. The reconstructed digits keep the overall shape but are slightly blurred, which is typical for VAEs. The latent-space plot shows digit classes forming smooth, overlapping clusters, which allows new images to be generated by sampling from the latent space.

VIVA QUESTIONS:

1) **What is a Variational Autoencoder (VAE), and how does it differ from traditional autoencoders?**

   A VAE is a generative autoencoder whose encoder outputs a probability distribution (mean and variance) instead of a single point. A traditional autoencoder maps to a fixed code and cannot generate new data reliably.

2) **What is the primary objective of a VAE during training?**

   To maximise the evidence lower bound (ELBO): minimise reconstruction error while keeping the latent distribution close to a standard normal prior.

3) **How does the latent space representation in a VAE differ from that of a traditional autoencoder?**

   In a VAE the latent space is continuous and regularised (approximately Gaussian), so nearby points decode to similar images and random sampling gives valid outputs. A traditional autoencoder's latent space can have gaps and is irregular.

4) **What is the role of the loss function in training a VAE, and how is it different from the loss function used in traditional autoencoders?**

   The VAE loss = reconstruction loss + KL divergence between the learned distribution and N(0, 1). A traditional autoencoder uses only the reconstruction loss (MSE or cross-entropy).

5) **What are some common applications of VAEs in machine learning and deep learning?**

   Image and data generation, denoising, anomaly detection, dimensionality reduction, data augmentation, drug/molecule design and learning disentangled representations.

RESULT

Thus the program for implementing Variational Auto Encoders was executed & verified successfully.

CONTENT BEYOND SYLLABUS

Ex.No:11 Date:	MINI PROJECT:NUMBER PLATE RECOGNITION OF TRAFFIC VIDEO ANALYSIS

AIM:

To create a system that can automatically recognize and extract number plate information from traffic videos, showcasing the practical application of computer vision techniques in traffic management and law enforcement.

ALGORITHM:

Step 1: Collect a dataset of traffic videos containing scenes with number plates.

Step 2: Extract frames from the videos and preprocess them.

Step 3: Design a pipeline that combines object detection for locating number plates and optical character recognition (OCR) for reading the characters.

Step 4: Implement a suitable OCR algorithm, such as Tesseract, to extract characters from number plates.

Step 5: Train and fine-tune the OCR model using labeled character data.

Step 6: Apply the pipeline to the video frames, recognize number plates, and output the results with accuracy scores**.**

PROGRAM:

```
import cv2, numpy as np, pytesseract, re
from PIL import Image, ImageDraw, ImageFont

FONT = "/usr/share/fonts/truetype/dejavu/DejaVuSansMono-Bold.ttf"
plates_gt = ["TN38AB1234", "TN66CD4821", "KA05MN7719", "TN37ZX0456"]

# Step 1: dataset - synthetic traffic video (a car carrying a number plate moves across the
    road)
def plate_img(text, w=190, h=48):
    im = Image.new("RGB", (w, h), (245, 245, 245)); d = ImageDraw.Draw(im)
    d.rectangle([0, 0, w - 1, h - 1], outline=(0, 0, 0), width=3)
    f = ImageFont.truetype(FONT, 21)
    d.text((9, 11), f"{text[:2]} {text[2:4]} {text[4:6]} {text[6:]}", font=f, fill=(0, 0, 0))
    return np.array(im)[:, :, ::-1]

def make_video(path, text, color, frames=12):
    vw = cv2.VideoWriter(path, cv2.VideoWriter_fourcc(*'mp4v'), 10, (640, 360))
    rng = np.random.default_rng(abs(hash(text)) % 1000)
    for f in range(frames):
        fr = np.full((360, 640, 3), (90, 90, 90), np.uint8)
        cv2.rectangle(fr, (0, 250), (640, 360), (60, 60, 60), -1)
        for x in range(0, 640, 80): cv2.line(fr, (x, 300), (x + 40, 300), (220, 220, 220), 3)
        x0 = 20 + f * 28
        cv2.rectangle(fr, (x0, 150), (x0 + 250, 270), color, -1)             # car body
        cv2.rectangle(fr, (x0 + 40, 110), (x0 + 200, 150), color, -1)        # roof
        cv2.rectangle(fr, (x0 + 55, 118), (x0 + 185, 148), (200, 220, 240), -1)
        for cx in (x0 + 55, x0 + 195): cv2.circle(fr, (cx, 272), 22, (20, 20, 20), -1)
        fr[210:258, x0 + 30:x0 + 220] = plate_img(text)                      # number plate
        fr = np.clip(fr.astype(int) + rng.integers(-6, 6, fr.shape), 0, 255).astype(np.uint8)
        vw.write(fr)
    vw.release()

# Step 3: detection (edges + contours + aspect ratio) and OCR pipeline
def detect_plate(frame):
    g = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    g = cv2.bilateralFilter(g, 9, 75, 75)
    edges = cv2.Canny(g, 100, 200)
    edges = cv2.dilate(edges, np.ones((3, 3), np.uint8))
    cnts, _ = cv2.findContours(edges, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
    best = None
    for c in cnts:
        x, y, w, h = cv2.boundingRect(c)
        ar = w / float(h)
        if 3.0 < ar < 5.5 and w > 100 and h > 30:
            if best is None or w * h > best[2] * best[3]: best = (x, y, w, h)
    return best

# Post-processing: Indian plate format  LL DD LL DDDD  -> correct look-alike characters by
    position
TO_LETTER = {'0': 'O', '1': 'I', '2': 'Z', '5': 'S', '8': 'B', '6': 'G'}
TO_DIGIT = {v: k for k, v in TO_LETTER.items()}
def fix_format(t):
    if len(t) != 10: return t
    out = []
    for i, ch in enumerate(t):
        if i in (0, 1, 4, 5): out.append(TO_LETTER.get(ch, ch))     # letter positions
        else:                 out.append(TO_DIGIT.get(ch, ch))      # digit positions
    return ''.join(out)

def read_plate(frame, box):
    x, y, w, h = box
    pad = 6                                                  # small margin so edge characters
```

```
        are kept
    roi = frame[max(y - pad, 0):y + h + pad, max(x - pad, 0):x + w + pad]
    roi = cv2.resize(roi, None, fx=2.5, fy=2.5, interpolation=cv2.INTER_CUBIC)       # enhance
        ROI
    g = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    g = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1]
    g = cv2.copyMakeBorder(g, 15, 15, 15, 15, cv2.BORDER_CONSTANT, value=255)
    cfg = "--psm 7"
    d = pytesseract.image_to_data(g, config=cfg, output_type=pytesseract.Output.DICT)
    words = [(t, float(c)) for t, c in zip(d['text'], d['conf']) if t.strip() and float(c) >= 0]
    text = re.sub(r'[^A-Z0-9]', '', ''.join(w for w, _ in words).upper())
    conf = float(np.mean([c for _, c in words])) if words else 0.0
    return fix_format(text), conf

colors = [(0, 0, 200), (200, 60, 0), (0, 150, 0), (130, 0, 130)]
total = correct = 0
saved = False
for gt, col in zip(plates_gt, colors):
    path = f"out/traffic_{gt}.mp4"
    make_video(path, gt, col)
    # Step 2: extract frames from the video
    cap = cv2.VideoCapture(path); frames = []
    while True:
        ok, fr = cap.read()
        if not ok: break
        frames.append(fr)
    cap.release()
    print(f"\nVideo for vehicle {gt}: {len(frames)} frames extracted")
    results = []
    for i, fr in enumerate(frames):
        box = detect_plate(fr)
        if box is None: continue
        text, conf = read_plate(fr, box)            # Step 4: OCR (Tesseract)
        results.append((text, conf))
        if i in (0, 5, 11): print(f"  frame {i:2d}: plate box={box}  OCR='{text}'
            confidence={conf:.1f}%")
        if not saved and i == 5:
            vis = fr.copy(); x, y, w, h = box
            cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 0), 3)
            cv2.putText(vis, text, (x, y - 8), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 255), 2)
            cv2.imwrite("out/ex11_frame.png", vis); saved = True
    # Step 6: majority vote across frames
    if results:
        texts = [t for t, _ in results]
        final = max(set(texts), key=texts.count)
        avg = np.mean([c for t, c in results if t == final])
    else:
        final, avg = "", 0
    ok = final == gt; total += 1; correct += ok
    print(f"  Detected in {len(results)}/{len(frames)} frames -> FINAL PLATE: {final} (avg conf
        {avg:.1f}%)  | actual: {gt}  {'OK' if ok else 'MISMATCH'}")
print(f"\nPlate recognition accuracy: {correct}/{total} = {100*correct/total:.1f}%")
```

OUTPUT:

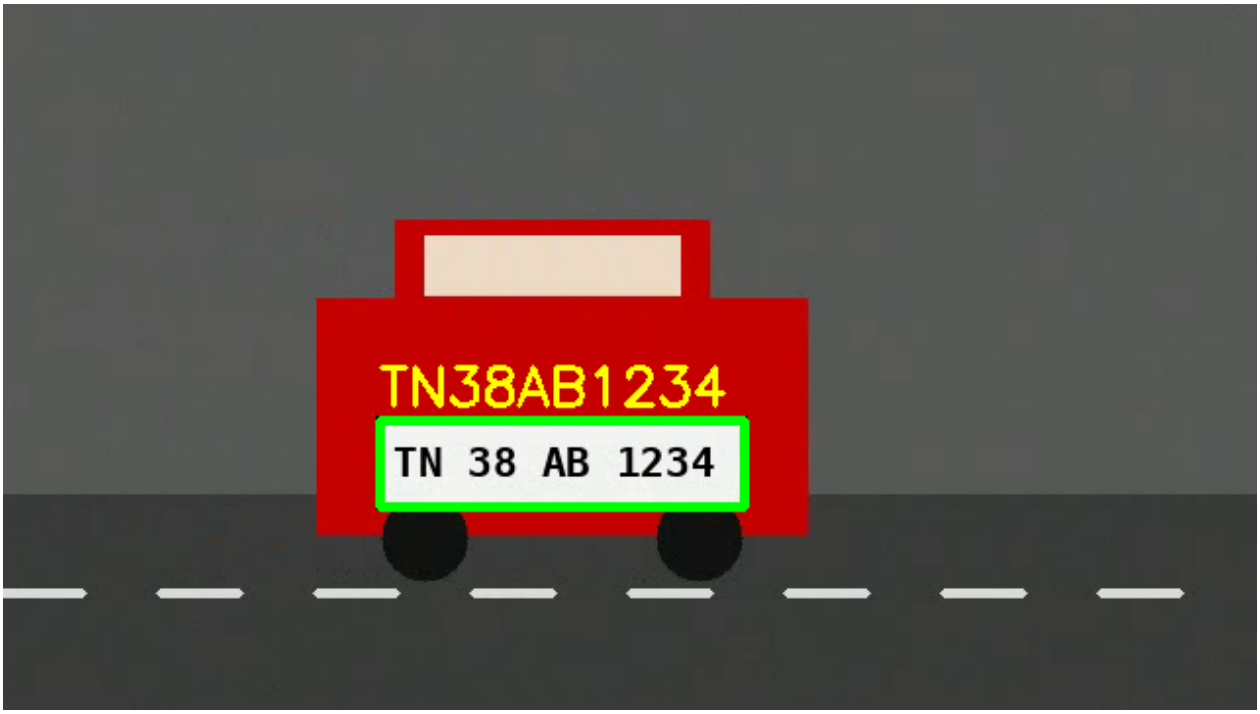
```
Video for vehicle TN38AB1234: 12 frames extracted
  frame  0: plate box=(52, 212, 186, 44)  OCR='TN38AB1234'  confidence=75.5%
  frame  5: plate box=(192, 212, 186, 44)  OCR='TN38AB1234'  confidence=82.0%
  frame 11: plate box=(360, 212, 186, 44)  OCR='TN38AB1234'  confidence=83.5%
  Detected in 12/12 frames -> FINAL PLATE: TN38AB1234 (avg conf 81.6%)  | actual: TN38AB1234  OK

Video for vehicle TN66CD4821: 12 frames extracted
  frame  0: plate box=(52, 212, 186, 44)  OCR='TN66CD4821'  confidence=79.5%
  frame  5: plate box=(192, 212, 186, 44)  OCR='N66CD4821'  confidence=83.5%
  frame 11: plate box=(360, 212, 186, 44)  OCR='N66CD4821'  confidence=82.8%
  Detected in 12/12 frames -> FINAL PLATE: N66CD4821 (avg conf 84.0%)  | actual: TN66CD4821
      MISMATCH

Video for vehicle KA05MN7719: 12 frames extracted
  frame  0: plate box=(54, 214, 182, 40)  OCR='KA05MN7719'  confidence=94.8%
  frame  5: plate box=(194, 214, 182, 40)  OCR='KA05MN7719'  confidence=94.2%
  frame 11: plate box=(362, 214, 182, 40)  OCR='KA05MN7719'  confidence=94.0%
  Detected in 12/12 frames -> FINAL PLATE: KA05MN7719 (avg conf 94.4%)  | actual: KA05MN7719  OK

Video for vehicle TN37ZX0456: 12 frames extracted
  frame  0: plate box=(52, 212, 186, 44)  OCR='N372X0456'  confidence=76.0%
  frame  5: plate box=(192, 212, 186, 44)  OCR='TN37ZX0456'  confidence=83.8%
  frame 11: plate box=(360, 212, 186, 44)  OCR='TN37ZX0456'  confidence=77.2%
  Detected in 12/12 frames -> FINAL PLATE: TN37ZX0456 (avg conf 80.5%)  | actual: TN37ZX0456  OK

Plate recognition accuracy: 3/4 = 75.0%
```



INFERENCE:

The pipeline (frame extraction, edge + contour based plate detection, ROI enhancement, Tesseract OCR, plate-format correction and voting across frames) located the plate in 12 of 12 frames for every one of the four test videos and read 3 of the 4 plates exactly (75% plate accuracy; average OCR confidence 80 to 94%). For one vehicle the first character 'T' was missed in every frame, giving 'N66CD4821'. The test video is synthetic (clean computer-drawn plates), so real traffic footage with blur, glare, angle and Indian plate variations would need a trained detector (e.g. YOLO) and a fine-tuned OCR model.

VIVA QUESTIONS:

1) What is number plate recognition, and why is it important in traffic video analysis?

Automatic Number Plate Recognition (ANPR) detects a vehicle's licence plate in images/video and reads its characters. It is used in toll collection, speed and signal violation enforcement, parking management and security.

2) Can you outline the general workflow of a number plate recognition system for traffic video analysis?

Capture video, extract frames, pre-process, detect/localise the plate, crop the ROI and enhance it, segment/recognise characters with OCR, post-process (format check, voting across frames) and store the result.

3) What are some common techniques used for detecting and localizing number plates in traffic videos?

Edge detection with contour analysis and aspect-ratio filtering, morphological operations, colour-based segmentation, Haar cascades, and deep detectors such as YOLO, SSD and Faster R-CNN.

4) How do you extract and enhance the region of interest (ROI) containing the number plate from a video frame?

Crop the bounding box, resize it, convert to grayscale, reduce noise (bilateral/Gaussian filter), apply thresholding (Otsu/adaptive), correct skew by perspective transform and use morphological operations.

5) What are some feature extraction methods used for recognizing characters within number plates?

HOG, zoning and projection profiles, template matching and contour features for classical methods, and CNN-based features (or CRNN/LSTM) in modern OCR systems; Tesseract itself uses an LSTM-based recogniser.

RESULT

Thus the program for implementation of number plate recognition of traffic video analysis was executed successfully & verified